

API Documentation

General Information

Field	Value
Base URL	<code>http://localhost:8000</code>
Data Format	JSON
Headers	<code>Content-Type: application/json</code>

Authentication

Type: JWT (JSON Web Token)

Mechanism:

1. User registers or logs in
2. Server returns a JWT token
3. Client stores the token
4. Client includes token in `Authorization` header for protected endpoints

Header Format:

```
Authorization: Bearer <token>
```

Endpoints

1. User Registration

Endpoint: `POST /auth/signup`

Description: Creates a new user account.

Request Body

Field	Type	Required	Description
first_name	string	Yes	User's first name
last_name	string	Yes	User's last name
email	string	Yes	User's email address
password	string	Yes	User's password

Example:

```
{
  "first_name": "John",
  "last_name": "Doe",
  "email": "john@example.com",
  "password": "SecurePass123"
}
```

Success Response (201 Created)

```
{
  "id": 1,
  "first_name": "John",
  "last_name": "Doe",
  "email": "john@example.com"
}
```

Error Responses

Status	Message	Description
400	"Please Login"	Email already registered
422	Validation Error	Invalid email format or missing fields

Business Logic

1. Validates email format
2. Checks if email already exists
3. Hashes password using Bcrypt

- 4. Creates user in database
 - 5. Returns user data (without password)
-

2. User Login

Endpoint: `POST /auth/login`

Description: Authenticates user and returns JWT token.

Request Body

Field	Type	Required	Description
email	string	Yes	User's email address
password	string	Yes	User's password

Example:

```
{
  "email": "john@example.com",
  "password": "SecurePass123"
}
```

Success Response (200 OK)

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Error Responses

Status	Message	Description
400	"Please create an Account"	Email not found
400	"Please check your Credentials"	Invalid password
500	"Unable to process request"	Server error

Business Logic

1. Retrieves user by email
 2. If user not found → returns 400
 3. Verifies password
 4. If invalid → returns 400
 5. Generates JWT token
 6. Returns token
-

3. Health Check

Endpoint: `GET /health`

Description: Verifies that the server is running.

Success Response (200 OK)

```
{  
  "status": "Running...."  
}
```

4. Protected Route

Endpoint: `GET /protected`

Description: Tests authentication and returns user data.

Headers

Header	Value	Required
<code>Authorization</code>	<code>Bearer <token></code>	Yes

Success Response (200 OK)

```
{
  "data": {
    "id": 1,
    "first_name": "John",
    "last_name": "Doe",
    "email": "john@example.com"
  }
}
```

Error Responses

Status	Message	Description
401	"Invalid Authentication Credentials"	Token missing, invalid, or expired

Business Logic

1. Validates **Authorization** header
2. Verifies token format (must start with **Bearer**)
3. Decodes and validates JWT token
4. Retrieves user by **user_id** from token
5. Returns user data

Data Schemas

UserInCreate (Registration)

Field	Type	Required	Description
first_name	string	Yes	User's first name
last_name	string	Yes	User's last name
email	string	Yes	User's email address
password	string	Yes	User's password

UserInLogin

Field	Type	Required	Description
email	string	Yes	User's email address
password	string	Yes	User's password

UserOutput (Response)

Field	Type	Description
id	integer	Unique user identifier
first_name	string	User's first name
last_name	string	User's last name
email	string	User's email address

UserWithToken (Login Response)

Field	Type	Description
token	string	JWT token

HTTP Status Codes

Code	Meaning	Description
200	OK	Request successful
201	CREATED	Resource created successfully
400	BAD_REQUEST	Invalid input data
401	UNAUTHORIZED	Authentication required or failed
422	UNPROCESSABLE_ENTITY	Validation error
500	INTERNAL_SERVER_ERROR	Server error

Security Notes

Password Storage

- Passwords are **never stored in plain text**
- **Algorithm:** Bcrypt with unique salt per password
- Hash format: `$2b$<cost>$<salt><hash>`

Token Security

- **Algorithm:** HS256 (HMAC-SHA256)
- **Expiration:** 900 seconds (15 minutes)
- **Transmission:** Always via `Authorization` header with `Bearer` prefix